UNIVERSIDAD CENTRAL · FACULTAD DE INGENIERÍA
Paradigmas y Técnicas de Programación · Unidad 3: Paradigma Paralelo

TALLER PREPARATORIO DEL PARCIAL · Python / Java

Fecha: _____	Nombres: _____	Equipo (máx. 2 personas)

Instrucciones: Resuelve los tres ejercicios en Python o Java. Ejecuta tu código, captura la salida y responde las preguntas de análisis. Se evalúa el código funcional y la comprensión del concepto. Entrega este taller con el código adjunto al finalizar la clase.

E1 Identifica y Corrige los Errores

Sincronización con hilos · Barrera join

20 min

Contexto

El siguiente programa intenta lanzar 4 hilos que simulan trabajo en paralelo y espera que todos terminen antes de imprimir el mensaje final. Contiene **4 errores** que impiden su correcto funcionamiento. Identifícalos, explica cada uno y escribe el código corregido.

Código con errores — Python

```python
import threading
import time

def trabajador(id_hilo, duracion):
    print(f"Hilo {id_hilo} iniciando...")
    time.sleep(duracion)
    print(f"Hilo {id_hilo} terminado.")

duraciones = [0.5, 0.3, 0.8, 0.2]
hilos = []

for i in range(4):
    # ERROR 1: Thread no recibe argumentos — falta args=(i, duraciones[i])
    t = threading.Thread(target=trabajador)
    hilos.append(t)

# ERROR 2: los hilos nunca se inician (falta t.start())
for h in hilos:
    pass

# ERROR 3: join() se llama antes de start() — orden incorrecto
for h in hilos:
    h.join()

# ERROR 4: este print podría ejecutarse antes de que los hilos arranquen
# (en este código el problema es que join sin start no espera nada útil)
```

```
print("Todos los hilos han terminado.")
```

Código con errores — Java (equivalente)

```
public class BarreraRota {
    static void trabajador(int id, int ms) throws InterruptedException {
        System.out.println("Hilo " + id + " iniciando...");
        Thread.sleep(ms);
        System.out.println("Hilo " + id + " terminado.");
    }
    public static void main(String[] args) throws Exception {
        int[] dur = {500, 300, 800, 200};
        Thread[] hilos = new Thread[4];
        for (int i = 0; i < 4; i++) {
            final int id = i, ms = dur[i];
            // ERROR 1: lambda llama trabajador con argumento incorrecto
            hilos[i] = new Thread(() -> trabajador(id));
        }
        // ERROR 2: start() nunca se llama
        for (Thread h : hilos) { /* vacío */ }

        // ERROR 3: join antes de start — no espera hilos reales
        for (Thread h : hilos) h.join();

        System.out.println("Todos los hilos han terminado.");
    }
}
```

Tu respuesta — Tabla de errores

#	Línea / Sección	Explicación del error y corrección
1		
2		
3		
4		

Código corregido (escribe aquí):

Preguntas de análisis

1	¿Qué sucede en tiempo de ejecución si un hilo no tiene start()? ¿El programa lanza error o continúa silenciosamente?
2	¿Por qué es obligatorio pasar args=(id, duracion) al crear el Thread? ¿Qué excepción genera Python al llamar trabajador() sin argumentos?
3	¿Por qué el join() debe llamarse después del start()? Explica qué ocurre si el orden es el contrario.

E2 Productor – Consumidor con Cola Compartida

Implementación · Sincronización con queue.Queue

25 min

Contexto

El patrón **Productor–Consumidor** coordina dos hilos: uno genera datos y los coloca en una cola compartida; el otro los lee y procesa. La sincronización la garantiza la cola thread-safe, sin necesidad de un Lock explícito.

Enunciado

1. Un hilo Productor genera 10 enteros aleatorios entre 1 y 50 (uno cada 0.1 s) y los pone en la cola. Al terminar, coloca el centinela -1.
2. Un hilo Consumidor lee de la cola en bucle. Si recibe -1, termina; si no, imprime el número y acumula la suma parcial.
3. El programa principal lanza ambos hilos, espera que terminen (join) e imprime la suma total.
4. Usa queue.Queue en Python o LinkedBlockingQueue<Integer> en Java.
5. Captura la salida de tu programa y pégala en el espacio indicado.

Código base — rellena los TODO (Python)

```
import threading, queue, random, time

cola = queue.Queue()  # cola thread-safe

def productor():
    for _ in range(10):
        num = random.randint(1, 50)
        # TODO 1: imprime '[Productor] Generado: <num>'
        # TODO 2: pon num en la cola
        time.sleep(0.1)
    # TODO 3: pon el centinela -1 en la cola para avisar al consumidor

def consumidor():
    acumulado = 0
    while True:
        item = ________________  # TODO 4: saca elemento de la cola (bloqueante)
        if item == -1:
            break
        acumulado += item
        # TODO 5: imprime '[Consumidor] Recibido: <item> | Suma: <acumulado>'
    print(f"Suma total: {acumulado}")

# TODO 6: crea hilos para productor y consumidor, inicialos y aplica join
```

Código base — Java (opcional)

```
import java.util.concurrent.*;
import java.util.Random;
public class ProdCons {
    static BlockingQueue<Integer> cola = new LinkedBlockingQueue<>();
    static void productor() throws InterruptedException {
        Random r = new Random();
        for (int i = 0; i < 10; i++) {
            int num = r.nextInt(50) + 1;
            // TODO 1: imprime y pon en cola (cola.put(num))
            Thread.sleep(100);
        }
        // TODO 2: pon centinela -1
    }
    static void consumidor() throws InterruptedException {
        int acum = 0;
        while (true) {
            int item = cola.________________;  // TODO 3: saca elemento bloqueante
```

```
            if (item == -1) break;
            acum += item;
            // TODO 4: imprime recibido y acumulado
        }
        System.out.println("Suma total: " + acum);
    }
    public static void main(String[] args) throws Exception {
        // TODO 5: crea e inicia los dos hilos; aplica join
    }
}
```

Pega aquí la salida de tu programa:

Preguntas de análisis

1	¿Por qué queue.Queue hace innecesario usar un Lock explícito? ¿Qué garantiza internamente esa clase?
2	¿El consumidor siempre recibe los números en el mismo orden en que el productor los generó? Justifica tu respuesta.
3	¿Qué pasaría si hubiera 2 consumidores y solo un centinela -1? ¿Cómo lo resolverías?

	_____
4	¿Qué ventaja tiene usar queue.get() bloqueante en lugar de un bucle activo (busy-wait) que compruebe si la cola está vacía? _____ _____ _____

E3 Pool de Hilos y Medición de Speedup

Implementación · ThreadPoolExecutor · Ley de Amdahl

20 min

Contexto

En lugar de crear hilos manualmente, los **pools de hilos** reutilizan un conjunto fijo de trabajadores. Python ofrece `ThreadPoolExecutor` y Java `ExecutorService`. Ambos permiten enviar tareas y recoger resultados con **Futures**.

Enunciado

1. Tienes una lista de 8 frases (dadas abajo). Cada frase debe ser procesada por contar_palabras(frase): (a) imprime qué worker la procesa, (b) simula trabajo con time.sleep(0.3), y (c) retorna el número de palabras.
2. Versión secuencial: procesa las 8 frases una por una con un bucle for. Mide el tiempo y acumula el total de palabras.
3. Versión paralela: usa ThreadPoolExecutor(max_workers=4), envía las tareas con submit() o map() y recoge los resultados. Mide el tiempo y verifica que el total coincida.
4. Calcula e imprime el speedup = T_secuencial / T_paralelo.

Lista de frases:

```
frases[0] = "El paradigma paralelo divide el trabajo entre múltiples hilos"
frases[1] = "Un hilo es la unidad mínima de ejecución en un proceso"
frases[2] = "La condición de carrera ocurre sin sincronización adecuada"
frases[3] = "El lock garantiza acceso exclusivo a la sección crítica"
frases[4] = "ThreadPoolExecutor reutiliza hilos para reducir el overhead"
frases[5] = "El speedup mide la ganancia de velocidad del programa paralelo"
frases[6] = "La Ley de Amdahl limita el speedup máximo teórico posible"
frases[7] = "El patrón productor consumidor usa una cola compartida segura"
```

Código base — rellena los TODO (Python)

```
import time, threading
from concurrent.futures import ThreadPoolExecutor

frases = [
    "El paradigma paralelo divide el trabajo entre múltiples hilos",
    "Un hilo es la unidad mínima de ejecución en un proceso",
    "La condición de carrera ocurre sin sincronización adecuada",
    "El lock garantiza acceso exclusivo a la sección crítica",
    "ThreadPoolExecutor reutiliza hilos para reducir el overhead",
    "El speedup mide la ganancia de velocidad del programa paralelo",
    "La Ley de Amdahl limita el speedup máximo teórico posible",
    "El patrón productor consumidor usa una cola compartida segura",
]
```

```
def contar_palabras(frase):
    # TODO 1: imprime qué worker procesa esta frase (threading.current_thread().name)
    time.sleep(0.3)
    # TODO 2: retorna el número de palabras de frase

# --- Versión SECUENCIAL ---
t0 = time.time()
total_seq = 0
for f in frases:
    # TODO 3: llama contar_palabras y acumula en total_seq
    pass
t_seq = time.time() - t0
print(f"Secuencial — palabras: {total_seq}  tiempo: {t_seq:.2f}s")

# --- Versión PARALELA ---
t0 = time.time()
# TODO 4: usa ThreadPoolExecutor(max_workers=4) y map() o submit()
total_par = ________________  # TODO 5: suma los resultados
t_par = time.time() - t0
print(f"Paralelo   — palabras: {total_par}  tiempo: {t_par:.2f}s")

speedup = ________________  # TODO 6: calcula speedup
print(f"Speedup: {speedup:.2f}x")
```

Registra tus tiempos:

T_secuencial (s)	T_paralelo (s)	Speedup = T_seq / T_par

Preguntas de análisis

1	¿El total de palabras coincide en la versión secuencial y paralela? ¿Por qué debe ser siempre igual aunque el orden de ejecución varíe?
2	¿El speedup obtenido fue cercano a 4x? Si no llegó a 4x, ¿qué factores según la Ley de Amdahl lo limitaron?
3	¿En qué se diferencia usar submit() de usar map() en el ThreadPoolExecutor? ¿Cuándo preferirías uno sobre el otro?

4	¿Este problema usa descomposición de dominio o descomposición funcional? Justifica con base en cómo se distribuye el trabajo. _____ _____ _____

Criterios de Evaluación

Criterio	Puntos
Ejercicio 1: errores identificados y explicados + código corregido + preguntas	25
Ejercicio 2: código funcional + salida capturada + preguntas respondidas	35
Ejercicio 3: código funcional + tiempos registrados + speedup + preguntas	30
Claridad del código, comentarios y trabajo en equipo	10
TOTAL	**100**